

Date of publication January 10, 2026, date of current version January 10, 2026.

Digital Object Identifier 10.1109/ACCESS.2017.Doi Number

SMART CONTRACT ANALYSIS (Final Report)

Furkan Filicioğlu and Enis Buğra Okunakol

Department of Computer Engineering, Gebze Technical University, Kocaeli, Türkiye

Corresponding author: Enis Buğra Okunakol (e-mail: e.okunakol2021@gtu.edu.tr)

This work didn't receive any grant from funding agencies

ABSTRACT Smart contracts play a critical role in blockchain based systems however, security vulnerabilities within these contracts have resulted in substantial financial losses in recent years. Although academic research has proposed various best practices for secure smart contract development, the extent to which these recommendations are applied in real world Ethereum contracts remains unclear. This study aims to investigate the gap between academic security guidelines and their practical adoption in deployed smart contracts. In this project, a hybrid analysis framework is presented that combines code-based security pattern detection with natural language processing techniques. First, Solidity smart contract codes collected from publicly available sources are analyzed to identify common risky patterns such as unsafe function calls and missing security mechanisms. Subsequently, project description texts are analyzed using text preprocessing and rulebased classification methods to distinguish between potentially malicious and legitimate contracts. In addition, academic articles related to smart contract security are examined using topic modeling techniques to identify frequently discussed vulnerabilities and recommended security practices. The findings indicate that while several security issues are extensively emphasized in the literature, their corresponding protective mechanisms are not sufficiently implemented in practice. The proposed system architecture enables a structured comparison between theoretical recommendations and real-world implementations. This study highlights the need for closer alignment between academic research and practical development and provides valuable insights for developers and researchers working in the field of blockchain security.

INDEX TERMS block chain security, natural language processing, code analysis, smart contracts

I. INTRODUCTION

Blockchain technology has gained significant attention in recent years due to its decentralized structure and transparent transaction model. One of the most important components of blockchain platforms is smart contracts, which are self-executing programs that automatically enforce predefined rules without the need for intermediaries. Ethereum is currently the most widely used platform for deploying smart contracts, enabling developers to build decentralized applications across various domains.

Despite their advantages, smart contracts are highly sensitive to security vulnerabilities. Even small coding mistakes can lead to serious exploits, resulting in large financial losses and loss of user trust. Several well known incidents have shown that once a vulnerable smart contract is deployed, it is extremely difficult to fix or update it. For this reason, smart contract security has become a major research topic in both academia and industry.

Previous studies have proposed various security mechanisms and development guidelines to reduce risks in

smart contracts. These include safe coding patterns, access control mechanisms, and defensive programming techniques. However, there is limited practical evidence showing whether these academic recommendations are actually followed in realworld smart contract implementations.

This project aims to analyze the gap between academic research and practical deployment of smart contract security measures. By combining code analysis with natural language processing techniques, this study investigates how frequently recommended security practices appear in deployed Ethereum contracts. The results of this analysis provide useful insights into current development practices and highlight areas where stronger alignment between theory and practice is needed.

II. LITERATURE REVIEW

Smart contract security has attracted increasing attention as blockchain-based systems have become more widely adopted. Many studies focus on identifying common vulnerabilities in Ethereum smart contracts, including reentrancy attacks, improper access control, and unsafe external calls. These works highlight that once smart contracts are deployed, they cannot be easily modified, which increases the importance of secure design before deployment.

Several researchers have proposed static analysis tools to detect security weaknesses directly from smart contract code. Pattern based methods are frequently used to identify risky structures such as the use of tx.origin, unchecked external calls, and missing event definitions. Other studies emphasize the role of standardized libraries and established development patterns in reducing implementation errors. While these approaches are effective for detecting known vulnerabilities, they often rely on predefined rules and may not fully capture more complex or context dependent risks.

III. BACKGROUND INFO

This section provides background information about the proposed system and its main components. The project focuses on analyzing smart contract security by combining code-level inspection with text-based analysis. The overall approach is designed to be modular, allowing different analysis components to operate independently while contributing to a unified evaluation framework.

A. The model the architecture

The proposed architecture consists of three main layers: data collection, analysis, and comparison. In the first layer, smart contract source codes, project description texts, and academic articles are collected from publicly available platforms. These data sources form the foundation of the analysis and are stored in a structured format for further processing.

The second layer performs the core analysis tasks. Smart contract codes are examined using pattern-based rules to identify potentially risky constructs and missing security mechanisms. In parallel, project description texts are processed through basic natural language preprocessing steps, such as lowercasing and removal of irrelevant terms, followed by rule-based classification to identify suspicious language patterns. Academic articles are analyzed separately to extract commonly discussed security topics and recommended practices.

In the final layer, the results obtained from code analysis, text analysis, and literature analysis are combined and compared. This comparison enables the identification of similarities and differences between theoretical recommendations and real-world implementations. The modular design of the architecture allows each component to be extended or replaced without affecting the overall system structure, making the framework flexible and scalable for future improvements.

Beyond codelevel analysis, recent research has explored the use of natural language processing techniques for blockchain security. Some studies analyze project documentation, smart contract descriptions, and online content to detect misleading claims or potential scam behavior. These approaches indicate that textual information can provide additional insight into the intent and trustworthiness of a project, especially when code analysis alone is not sufficient.

Academic literature also frequently discusses best practices such as access control mechanisms, defensive programming strategies, and secure development frameworks. However, most existing research primarily focuses on proposing new detection techniques rather than examining how widely these recommended practices are adopted in deployed smart contracts. This observation suggests that further analysis is needed to better understand the relationship between academic recommendations and real-world implementation practices.

IV. METHODOLOGY

This section describes the methodology used to analyze smart contract security and compare academic recommendations with real-world implementations. The proposed approach follows a structured pipeline consisting of data collection, preprocessing, analysis, and comparison.

A. Data Collection

Data are collected from three primary sources. First, verified Ethereum smart contract source codes are obtained from publicly available repositories. Second, project description texts associated with these contracts are gathered to support textual analysis. Third, academic articles related to smart contract security are collected from scientific databases to represent theoretical recommendations. All data are stored in a structured format to ensure consistency during analysis.

I. B. Code Analysis

Smart contract codes are analyzed using a pattern-based approach. A predefined list of risky constructs is used to detect potential security issues, such as unsafe function calls and missing protection mechanisms. Each contract is scanned for these patterns, and the findings are recorded to represent the security posture of the code. This approach enables systematic inspection without modifying the original contracts.

II. C. Text Analysis

Project description texts undergo basic preprocessing steps, including lowercasing and removal of irrelevant terms. After preprocessing, the texts are evaluated using rule-based criteria to identify suspicious language patterns that may indicate risky or misleading behavior. This textual analysis complements code inspection by providing contextual information that may not be visible at the code level.

III. D. Literature Analysis and Comparison

Academic articles are analyzed to identify commonly discussed security vulnerabilities and recommended practices. The extracted findings are then compared with the results

obtained from code and text analysis. This comparison allows the evaluation of how frequently academic recommendations appear in deployed smart contracts, and highlights differences between theoretical guidance and practical implementation.

V. IMPLEMENTATION

This section describes the implementation process and presents the initial results obtained from the proposed system. The implementation focuses on applying the designed analysis pipeline to a real dataset of Ethereum smart contracts and their associated textual descriptions. The system is implemented using Python and standard data processing libraries, allowing reproducible and consistent analysis.

During implementation, smart contract source codes are processed individually and scanned for predefined risky patterns. The detection results are recorded for each contract, including the type and frequency of identified security related constructs. In parallel, project description texts are processed and evaluated using rulebased criteria to identify potentially misleading or suspicious expressions. These two analysis outputs are stored separately and later combined for comparison.

Initial results indicate that several risky coding patterns frequently discussed in academic studies still appear in deployed smart contracts. In particular, missing protection mechanisms and unsafe external interactions are observed in a notable portion of the analyzed contracts. Text analysis results show that some project descriptions contain overly optimistic or vague statements that may raise concerns regarding transparency and reliability.

The combined analysis provides a clearer view of how theoretical security recommendations align with practical implementations. While some widely recommended practices are observed, others appear to be inconsistently applied. These findings suggest that although awareness of smart contract security has increased, there is still room for improvement in adopting established best practices during development.



FIGURE 1 Real-world smart contract exploits and their financial impact. The figure highlights major security incidents, including reentrancy attacks, logic flaws, and scam-based contracts, demonstrating the significant economic losses caused by vulnerabilities in deployed smart contracts.

VI. RESULTS & DISCUSSION

This section presents the results obtained from the implementation and discusses their implications in relation to existing academic studies. The analysis is based on the combined evaluation of smart contract code inspection and textual assessment of project descriptions.

The results from code analysis indicate that several commonly known security weaknesses are still present in deployed smart contracts. Risky patterns related to external calls and missing protection mechanisms are observed in a noticeable portion of the analyzed contracts. Although some contracts apply basic security practices, advanced defensive mechanisms appear to be inconsistently implemented across the dataset.

The text analysis results reveal that a number of project descriptions contain vague or overly optimistic statements. Such language may reduce transparency and make it difficult for users to accurately assess the reliability of a project. While not all suspicious textual patterns directly indicate malicious intent, their presence raises concerns when combined with weak code-level security practices.

When the findings are evaluated together, a partial alignment between academic recommendations and real-world implementations can be observed. Certain widely discussed security practices appear in deployed contracts, whereas others are rarely applied despite their frequent emphasis in the literature. This difference suggests that practical constraints, limited developer expertise, or time pressure may influence security-related decisions during development.

Overall, the results highlight that awareness of smart contract security has increased, yet the adoption of established best practices remains uneven. These findings emphasize the importance of integrating security considerations more effectively into the development lifecycle of smart contracts.

VII. CONCLUSION

This study presented an analysis of smart contract security by combining code-level inspection with textual evaluation and academic literature review. The findings demonstrate that, despite increased awareness of security risks, several widely known vulnerabilities and weak practices continue to appear in deployed smart contracts. This indicates that the transition from academic recommendations to practical implementation is still incomplete.

The results underline the importance of adopting security measures early in the development lifecycle, rather than treating security as a secondary concern. By jointly analyzing smart contract code and project-related textual information, the proposed approach provides a broader perspective on security assessment compared to methods that rely on a single data source.

This work can support developers, researchers, and auditors in better understanding current security practices and identifying areas that require improvement. Future work may extend this

study by analyzing larger datasets, integrating automated analysis techniques, or applying the framework to other blockchain platforms. Such extensions could contribute to improving the overall reliability and trustworthiness of smart contract-based systems.

ACKNOWLEDGMENT

The authors would like to thank their course instructor(Salih Sarp) for guidances and feedback provided during the development of this project

REFERENCES

- [1] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, 2014.
- [2] N. Atzei, M. Bartoletti, and T. Cimoli, "A survey of attacks on Ethereum smart contracts," in *Proc. Int. Conf. Principles of Security and Trust*, 2017, pp. 164–186.
- [3] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in *Proc. ACM Conf. Computer and Communications Security*, 2016, pp. 254–269.
- [4] C. Liu, H. Liu, Z. Cao, Z. Chen, B. Chen, and B. Roscoe, "Reentrancy vulnerability analysis of Ethereum smart contracts," *IEEE Access*, vol. 6, pp. 68098–68109, 2018.
- [5] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, "SmartCheck: Static analysis of Ethereum smart contracts," in *Proc. IEEE Int. Workshop on Blockchain Oriented Software Engineering*, 2018, pp. 9–16.
- [6] M. Bartoletti and L. Pompianu, "An empirical analysis of smart contracts: Platforms, applications, and design patterns," in *Proc. Financial Cryptography and Data Security*, 2017, pp. 494–509.
- [7] Y. Chen, H. Li, K. Chen, and X. Li, "Detecting scam smart contracts on Ethereum using data analysis," *IEEE Access*, vol. 7, pp. 77838–77847, 2019.
- [8] J. Zhou, X. Wang, Y. Guo, and S. Wang, "Detecting deceptive project descriptions in blockchain ecosystems," *Future Generation Computer Systems*, vol. 107, pp. 770–781, 2020.